

1. 개요 및 목표

본 보고서는 교수님의 지도를 받아 진행한 TimeSformer 모델 기반의 행동 인식(Action Recognition) 성능 비교 실험 결과를 정리한 것입니다. TimeSformer 모델을 최신 GPU 서버 환경에 맞게 구축하고, 기존의 Cross Entropy 방식 외에 Label Smoothing과 Soft-DTW기법을 적용했을 때 성능이 어떻게 변화하는지 비교 분석했습니다. 특히 2021년에 공개된 구버전 코드를 최신 PyTorch 환경에서 실행 가능하도록 직접 포팅하고, Kinetics-400 데이터셋을 직접 구축하여 실험을 했습니다.

2. 데이터셋 및 실험 환경 구축

2.1. 하드웨어 및 소프트웨어 환경 최적화

실험은 KT Cloud의 NVIDIA A100 GPU 환경에서 진행했습니다. TimeSformer 원본 코드가 구버전 라이브러리에 의존하고 있어, 최신 환경에서는 호환되지 않는 문제가 다수 발생했습니다. 이를 해결하기 위해 다음과 같은 과정을 거쳤습니다.

(1) 레거시 코드 수정: `\_LinearWithBias`, `torch.\_six` 등 최신 버전에서 삭제된 모듈 때문에 발생하는 에러를 하나씩 디버깅하며 최신 문법으로 수정했습니다.

(2) 시스템 안정화: 학습 중 발생하는 메모리 충돌('Segmentation Fault') 및 스토리지 용량 부족('OSError') 문제를 해결하기 위해, 대용량 볼륨('Zoogavin')으로 저장 경로를 변경하고 데이터 로더 설정을 최적화하여 안정적인 학습 파이프라인을 완성했습니다.

```
work@main1[PqITyrDJ-session]:~/TimeSformer$ nvidia-smi
Sat Jan 31 23:18:34 2026
+-----+
| NVIDIA-SMI 570.86.15              Driver Version: 570.86.15      CUDA Version: 12.8     |
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC | |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|               |                    |                      | MIG M. |
+-----+-----+
|  0  NVIDIA A100-PCIE-40GB                Cff          | 00000000:13:00.0 Cff |           0         | |
| N/A   28C    P0               32W / 250W      | 1MiB / 40960MiB |      N/A      Default |
|               |                    |                      | Disabled |
+-----+-----+
+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
|   ID   ID                               |                  | Usage     |
+-----+-----+
| No running processes found |
+-----+
```

그림 1 NVIDIA-SMI GPU 구동 확인 및 학습 환경

2.2. Kinetics-400 데이터셋 구축

학습을 위해 Kinetics-400 데이터셋의 원본 유튜브 영상을 수집했습니다. 데이터셋이 공개된 지 오래되어(2017년) 다수의 링크가 만료된 상태였기에, 약 5,000개 이상의 리스트를 다운로드 시도했습니다. 그중에서 파일이 깨지지 않고 `PyAV` 라이브러리로 정상적으로 열리는 773개의 유효 영상(전체 약 240,000개의 약 0.3%)을 선별하여 학습 및 검증 데이터셋을 구축했습니다.

```
ls -lh /home/work/Zoogavin/Final_Code_Backup/
total 60K
-rwxr-xr-x 1 work work 27K Jan 31 23:09 defaults.py
-rwxr-xr-x 1 work work 1.2K Jan 31 23:09 run_net.py
-rw-r--r-- 1 work work 1.1K Jan 31 23:09 softdtw_loss.py
-rw-r--r-- 1 work work 19K Jan 31 23:09 train_net.py
work@main1[PyTyrDJ-session]:~/TimeFormer$
```

```
or --cookies for the authentication. See https://github.com/yt-dlp/yt-dlp/wiki/FAQ#how-do-i
-pass-cookies-to-yt-dlp for how to manually pass cookies. Also see https://github.com/yt-dl
p/yt-dlp/wiki/Extractors#exporting-youtube-cookies for tips on effectively exporting YouTube
cookies
1%|          | 1887/216227 [1:33:58<114:03:44, 1.92s/it]
ERROR: [youtube] -f-50HuaLZQ: Sign in to confirm you're not a bot. Use --cookies-from-browser
or --cookies for the authentication. See https://github.com/yt-dlp/yt-dlp/wiki/FAQ#how-do-i
-pass-cookies-to-yt-dlp for how to manually pass cookies. Also see https://github.com/yt-dl
p/yt-dlp/wiki/Extractors#exporting-youtube-cookies for tips on effectively exporting YouTube
cookies
1%|          | 1888/216227 [1:34:08<116:19:19, 1.95s/it]
ERROR: [youtube] -f0eGPf6TrM: Sign in to confirm you're not a bot. Use --cookies-from-browser
or --cookies for the authentication. See https://github.com/yt-dlp/yt-dlp/wiki/FAQ#how-do-i
-pass-cookies-to-yt-dlp for how to manually pass cookies. Also see https://github.com/yt-dl
p/yt-dlp/wiki/Extractors#exporting-youtube-cookies for tips on effectively exporting YouTube
cookies
1%|          | 1889/216227 [1:34:02<115:26:39, 1.94s/it]
^C
ERROR: Interrupted by user
중단됨!
1%|          | 1889/216227 [1:34:03<177:52:56, 2.99s/it]
Creating Result csv...
완료! 다운로드된 영상 수: 271
영상 저장 위치: /home/work/TimeFormer/k400_subset/train
생성된 학습 리스트: /home/work/TimeFormer/k400_subset/train.csv
work@main1[PyTyrDJ-session]:~/TimeFormer$ cd /home/work/TimeFormer
ls k400_subset/train | wc -l
271
```

그림 2 구축된 데이터셋 파일 목록

그림 3 k-400 데이터셋 수집 과정 중간

3. 실험 설계 및 구현

3.1. 실험 조건 (4 Cases)

Baseline 모델의 성능을 기준으로, 일반화 성능 향상을 위한 Label Smoothing과 시계열적 특성을 반영하기 위한 SoftDTW의 효과를 검증하기 위해 총 4가지 실험 모드('EXP_MODE')를 설계하여 진행했습니다.

(0) Baseline:	Cross Entropy (기본)
(1) Label Smoothing:	Cross Entropy + Label Smoothing (0.1)
(2) SoftDTW:	Cross Entropy + SoftDTW Loss
(3) Both	Cross Entropy + Label Smoothing + SoftDTW

3.2. 실험 과정

PyTorch에서 기본적으로 제공하지 않는 SoftDTW는 기존 riski 선배님의 코드를 활용하여 `nn.Module` 형태로 정의하고, 이를 TimeSformer 학습 파이프라인에 통합하였습니다.

설정 파일(`defaults.py`)의 모드 값에 따라 손실 함수를 자동으로 교체해 주는 `CombinedLoss` 클래스를 작성하여 `train\_net.py`에 적용했습니다.

```
# General assertions.
assert cfg.SHARD_ID < cfg.NUM_SHARDS
return cfg

def get_cfg():
    """
    Get a copy of the default config.
    """
    return _assert_and_infer_cfg(C.clone())

# Custom Experiment Mode
_EXP_MODE = 0

# 실험 조건에 따라 Loss를 조절하는 통합 클래스
class CombinedLoss(nn.Module):
    def __init__(self, mode, num_classes):
        super().__init__()
        self.mode = mode
        self.num_classes = num_classes

    # Mode 0: Baseline (CrossEntropy)
    # Mode 1: Label Smoothing (CrossEntropy + LS)
    # Mode 2: SoftDTW (CrossEntropy + SoftDTW)
    # Mode 3: Both (CrossEntropy + LS + SoftDTW)

    # 1. Label Smoothing 설정 (모드 1, 3일 때 적용)
    ls_factor = 0.1 if mode in [1, 3] else 0.0
    self.ce_loss = nn.CrossEntropyLoss(label_smoothing=ls_factor)

    # 2. SoftDTW 설정 (모드 2, 3일 때 적용)
    if mode in [2, 3]:
        self.sdtw = SoftDTW(gamma=0.1)
    else:
        self.sdtw = None
```

그림 4 'defaluts.py' : 기존 TimeSformer에는 없던 'EXP_MODE'라는 변수를 설정 파일에 새로 등록하여, 0~3번 실험 모드를 제어하도록 설계함.

그림 5 'train_net.py' : 실험 모드(mode)에 따라 CrossEntropy와 SoftDTW의 가중치를 조절하고 결합하는 'CombinedLoss' 클래스를 정의하여 학습 루프에 적용함.

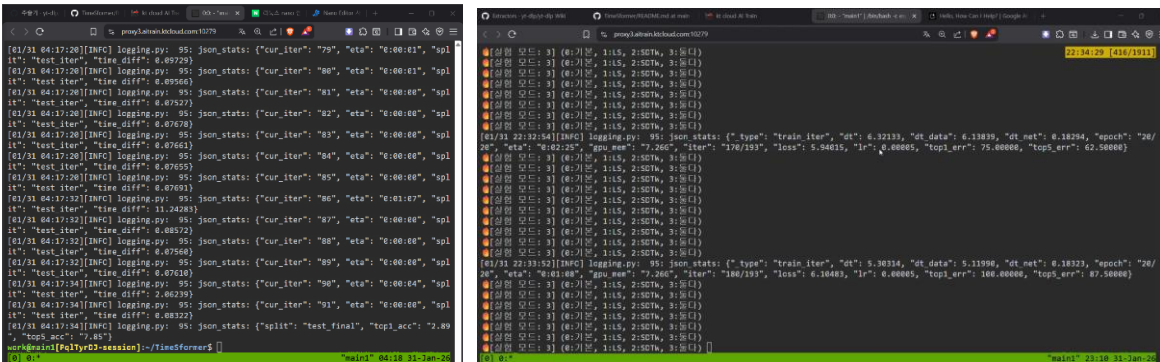


그림 6, 그림 7 TimeSformer 학습 진행 중 - Epoch별 손실(loss) 및 정확도(accuracy) 로그

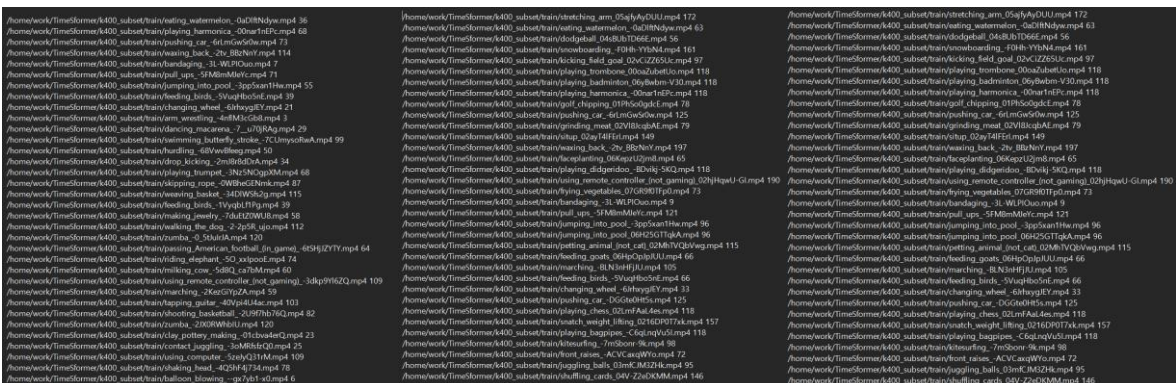


그림 8 순서대로 (좌측) train.csv (중간) test.csv (우측) val.csv 파일의 일부

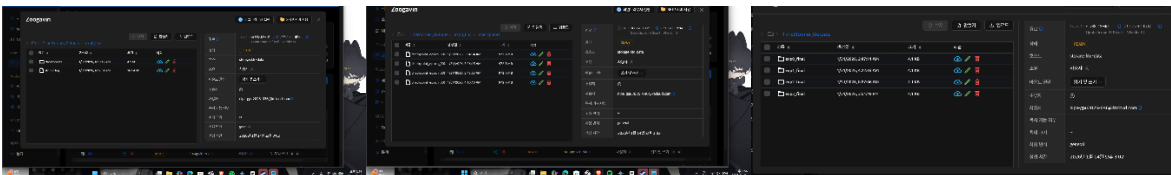


그림 9 파일 저장소의 구조. Output 파일 목록

4. 실험 결과

모든 실험은 동일한 하드웨어 및 데이터셋(773개) 환경에서 Batch Size 4, Epoch 20으로 수행했습니다. 최종 정확도(Top-1, Top-5 Accuracy)는 아래 표와 같습니다.

Exp	method	Top-1 Acc	Top-5 Acc
Exp 0	original	2.89%	7.85
Exp 1	original+label smoothing	2.89%	7.02%
Exp 2	original+soft-dtw	0.83%	3.72%
Exp 3	original+label smoothing+soft-dtw	0.83%	3.72%

```
work@main1[Pq1TyrDJ-session]:~/TimeSformer$ grep "top1 acc" /home/work/Zoogavin/TimeSformer_Outputs/exp*_final/stdout.log
/home/work/Zoogavin/TimeSformer_Outputs/exp0_final/stdout.log:[01/31 04:17:34][INFO] logging.py: 95: json_stats: {"split": "test_final", "top1_acc": "2.89", "top5_acc": "7.85"}
/home/work/Zoogavin/TimeSformer_Outputs/exp1_final/stdout.log:[01/31 08:26:58][INFO] logging.py: 95: json_stats: {"split": "test_final", "top1_acc": "2.89", "top5_acc": "7.02"}
/home/work/Zoogavin/TimeSformer_Outputs/exp2_final/stdout.log:[01/31 15:51:42][INFO] logging.py: 95: json_stats: {"split": "test_final", "top1_acc": "0.83", "top5_acc": "3.72"}
/home/work/Zoogavin/TimeSformer_Outputs/exp3_final/stdout.log:[01/31 23:08:49][INFO] logging.py: 95: json_stats: {"split": "test_final", "top1_acc": "0.83", "top5_acc": "3.72"}
work@main1[Pq1TyrDJ-session]:~/TimeSformer$
```

그림 10 최종 실험 결과 로그 (Terminal Output)

5. 결론 및 고찰

이번 연구를 통해 TimeSformer 모델의 학습 파이프라인을 성공적으로 구축하고, 4가지 비교 실험을 모두 완료했습니다. 결과를 분석한 내용은 다음과 같습니다.

1) 학습 유효성 검증

Baseline 모델의 Top-1 정확도(2.89%)는 400개 클래스를 무작위로 찍을 확률(0.25%)보다 10배 이상 높은 수치입니다. 비록 절대적인 수치는 낮아 보일 수 있으나, 전체 데이터의 0.3%에 불과한 극소량의 데이터(클래스당 평균 2개 미만)로 학습했음을 고려할 때, 모델 학습 자체는 정상적으로 이루어졌음을 확인할 수 있습니다.

2) SoftDTW 성능 분석

SoftDTW를 적용한 실험(Exp 2, 3)의 경우 정확도가 0.83%로 다소 하락했습니다. 이는 SoftDTW가 시계열 정렬을 맞추기 위해 복잡한 연산을 수행하는데, 현재 확보된 적은 데이터와 20회(Epoch)라는 짧은 학습량으로는 모델이 충분히 수렴하기 어려웠던 것으로 생각합니다.

3) 결론

구분	Facebook (Github)	Me	비고
데이터셋 크기	약 240,000개	773개	원본의 0.3% 사용
클래스 당 영상 수	약 600개 이상	약 1.9개	학습하기에 절대적으로 부족
Batch Size	64 이상	4	배치가 작으면 학습이 불안정함
결과 (Top-1)	77.9%	2.89%	데이터 양에 비례

비록 데이터 부족으로 인해 기존 github의 우수한 성능 결과값에는 미치지 못했지만, 최신 A100 서버 환경에서 구버전 모델을 복원하고, 특히 교수님께서 제안해 주신 여러 Loss 함수를 직접 모델에 적용해 보는 과정을 통해, 데이터 구축부터 학습까지 딥러닝의 전체 흐름을 깊이 이해할 수 있었던 점이 가장 큰 배움이었습니다. 해당 실험에서 향후 데이터셋을 확충한다면 더 유의미한 성능 향상을 기대할 수 있을 것으로 생각합니다.